

Introduction to Entity Framework Core

Welcome to our Entity Framework Core (EF Core) Tutorials. This Entity Framework Core (EF Core) Tutorial series will cover most of the concepts related to Entity Framework. We will start with the basics and end with the most advanced and new features provided by EF Core. In this article, I will briefly introduce the **Entity Framework Core** and discuss the following pointers.

1. [What is Entity Framework Core?](#)
2. [What is ORM?](#)
3. [Why Do We Need to Use An ORM?](#)
4. [Why Entity Framework Core in .NET Applications?](#)
5. [EF Core Development Approaches](#)
6. [What are EF Core Database Providers?](#)
7. [Why We Need to Use Entity Framework Core over EF 6.x?](#)

What is Entity Framework Core?

Entity Framework Core (EF Core) is an open-source, lightweight, extensible, and cross-platform version of Microsoft's popular Entity Framework data access technology. It is designed to work with .NET Core and .NET Framework applications and provides an Object-Relational Mapper (ORM) that enables .NET developers to work with a database using .NET objects. It eliminates most of the data-access code that developers usually need to write.

What is ORM?

ORM stands for Object-Relational Mapping, a programming technique that allows developers to convert data between incompatible systems, specifically between Object-Oriented Programming Languages (such as C#, Java, etc.) and Relational Databases (such as SQL Server, MySQL, Oracle, etc.).

ORM allows developers to work with data in terms of objects rather than tables and columns. That means ORM automatically creates classes based on database tables, and vice versa is also true. It can also generate the necessary SQL to create the database based on the classes.

As developers, we mainly work with data-driven applications, and the ORM Framework can generate the necessary SQL (to perform the database CRUD operation) that the underlying database can understand. It takes responsibility for opening the connection, executing the command, handling the transaction to ensure data integrity, closing the connection, etc. So, in simple words, we can say that the ORM Framework eliminates the need for most of the data access code we generally write.

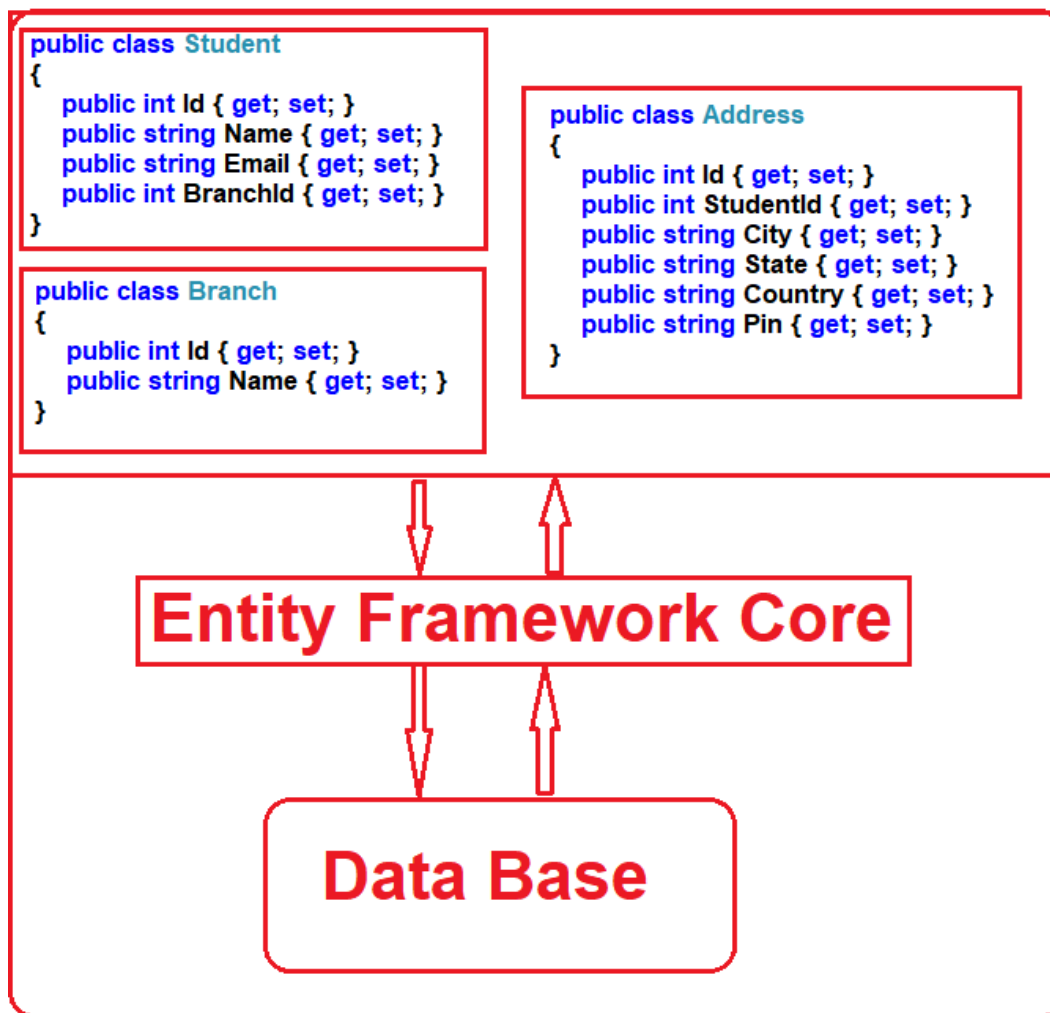
Why Do We Need to Use An ORM Framework?

Let's understand why we need to use the ORM Framework with an example. Suppose we want to develop an application to manage the students of a college. To do this, we may need to create classes such as Student, Department, Address, etc. Technically, we call these classes Domain classes or business objects.

Without using an ORM Framework like Entity Framework or EF Core, we have to write lots of data access code to perform the CRUD operations, i.e., store and retrieve the Student, Department, and Address data from the underlying database tables.

For example, to perform CRUD operations, i.e., read, insert, update, or delete from a database table, we generally need to write the required SQL statements, which the underlying database should understand. Again, when we want to read the data from the database into our application, we also have to write some custom logic to map the data to our model classes like Student, Department, Address, etc. This is a common task that we do in almost every application.

An ORM Framework like Entity Framework or EF Core can do all of the above for us and saves a lot of time if we provide the required information to the ORM Framework. The ORM Framework sits between our application code and the Database. It eliminates the need for most custom data-access codes we usually write without an ORM. For a better understanding, please have a look at the following diagram:



Why Should We Use ORM, like Entity Framework or Entity Framework Core, in our applications?

The following are some reasons to use an ORM like EF or Entity Framework Core in your applications.

- **Simplifies Data Access:** ORMs abstract the complexities of data access, allowing developers to interact with the database in an object-oriented rather than writing complex SQL queries.
- **Reduces Repetitive Code:** ORM frameworks automatically generate SQL queries, handle connection management, and perform CRUD operations, reducing the amount of repetitive code developers need to write.
- **Increased Productivity:** Developers can build applications faster by focusing on the application logic rather than data access details.
- **Cross-Platform Compatibility:** ORM frameworks like EF Core work across different platforms and databases, making it easier to develop cross-platform applications.
- **Increased Performance:** ORMs support Lazy Loading, Eager Loading, and Caching to optimize data retrieval operations.
- **Data Integrity:** ORM frameworks include capabilities such as concurrency handling and transactions that help maintain data integrity without extra effort from developers.

Why Entity Framework Core in .NET Applications?

EF Core is widely used in .NET applications for several reasons. They are as follows:

- **Cross-Platform:** EF Core works on various platforms, including Windows, macOS, and Linux.
- **Extensible:** EF Core is highly customizable and extensible, allowing developers to use custom conventions, third-party plugins, and additional database providers.
- **Performance:** EF Core has been optimized for performance, making it suitable for small, medium, and large-scale applications.
- **Integration:** EF Core integrates easily with .NET Core, making it the first choice for data access in .NET applications.
- **Flexibility:** EF Core supports various database providers, which allows it to work with different database systems.
- **Modern Features:** EF Core includes features like asynchronous operations, better support for LINQ queries, and improved support for complex types and relationships.

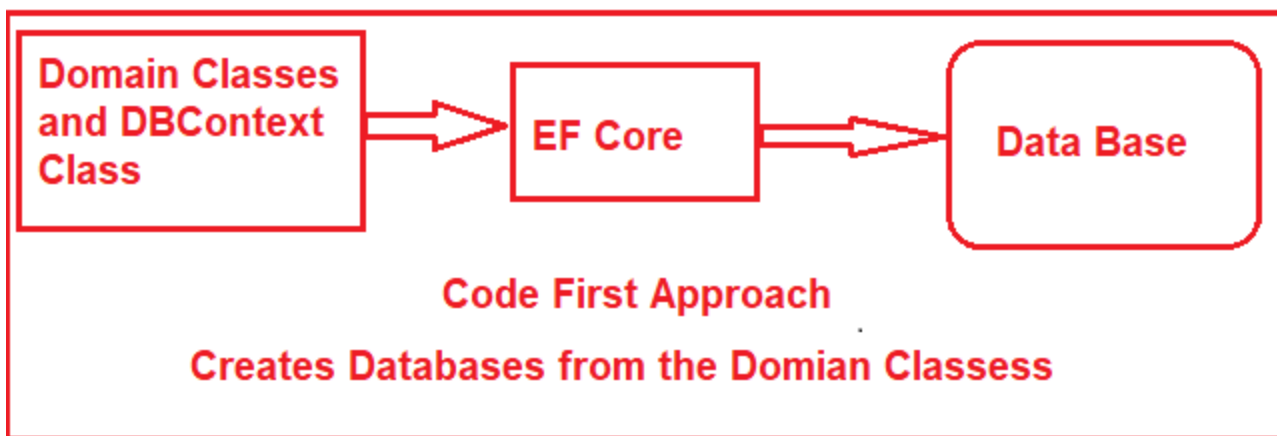
Entity Framework Core Development Approaches:

Entity Framework Core supports two development approaches. They are as follows:

1. **Code-First Approach:** In this approach, the data model (classes) is created first, and Entity Framework Core generates the database schema based on the model.
2. **Database-First Approach:** This approach is used when an existing database is available. Entity Framework Core can generate the data model (classes) based on the database schema.

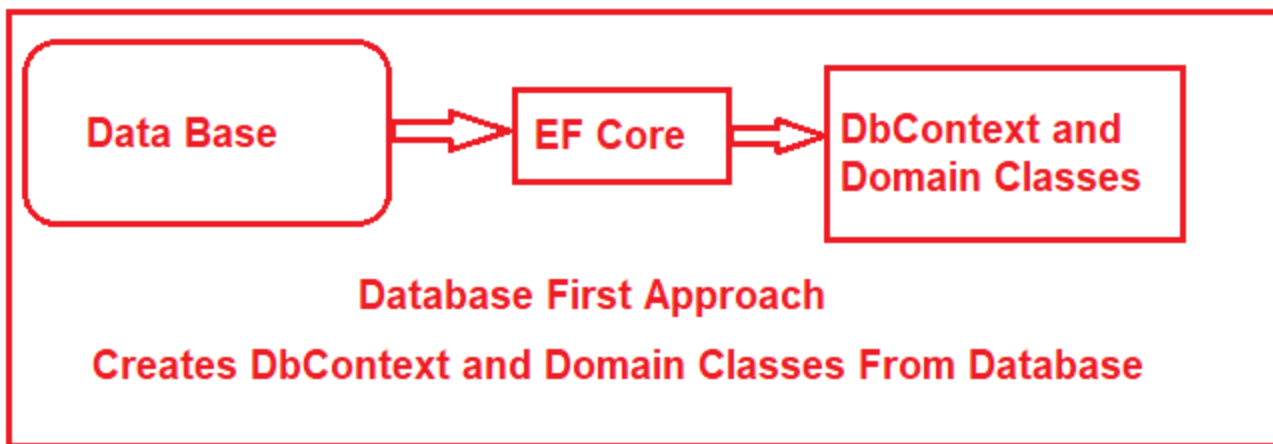
EF Core Code First Approach:

In the EF Core Code First Approach, first, we need to create our application domain classes, such as Student, Branch, Address, etc., and a special class (called DbContext Class) that derives from the Entity Framework Core DbContext class. Developers can configure relationships, constraints, and database mappings using Data Annotations and Fluent API. Then, based on the application domain classes and DbContext class, the EF Core creates the database and related tables. For a better understanding, please have a look at the following diagram.



EF Core Database First Approach:

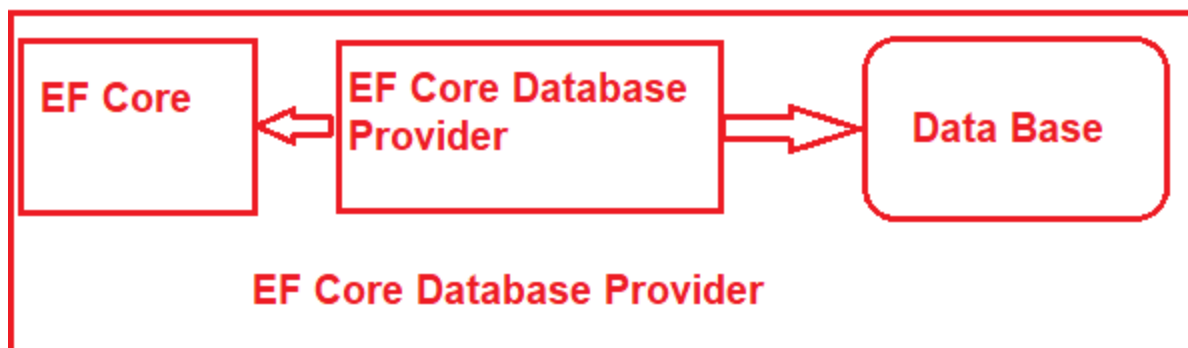
Use the EF Core Database First Approach if you have an existing database and database tables are already there. In the database-first approach, the EF Core creates the DbContext and Domain Classes based on the existing database schema. This approach is suitable for scenarios where the application code does not control the database schema or when working with existing databases. For a better understanding, please have a look at the following diagram.



So, the Database First approach starts with an existing database schema. EF Core generates the domain classes that map to the database tables. This approach is suitable for applications that need to work with an existing database.

EF Core Database Providers:

EF Core can work with various relational and non-relational databases such as Microsoft SQL Server, MySQL, PostgreSQL, SQLite, and more. This is possible because of the EF Core Database providers that are typically implemented as NuGet packages that can be added to the project to enable support for specific databases. The Database Provider sits between the EF Core and the Database it supports. For a better understanding, please look at the following diagram:



EF Core supports various database providers that allow it to work with different types of databases, such as:

- **Microsoft SQL Server:** The default provider for EF Core, which is widely used in enterprise applications. The NuGet Package for Microsoft SQL Server is **Microsoft.EntityFrameworkCore.SqlServer**
- **SQLite:** A lightweight, file-based database ideal for small-scale applications and prototyping. The NuGet Package for SQLite is **Microsoft.EntityFrameworkCore.Sqlite**
- **MySQL:** An open-source relational database that is popular in web applications. The NuGet Package for MySQL is **Pomelo.EntityFrameworkCore.MySql**
- **PostgreSQL:** An advanced, open-source relational database known for its robustness and performance. The NuGet Package for PostgreSQL is **Npgsql.EntityFrameworkCore.PostgreSQL**
- **InMemory:** A non-persistent, in-memory database provider mainly used for testing purposes. The NuGet Package for In-Memory Database is **Microsoft.EntityFrameworkCore.InMemory**.
- **Oracle:** A powerful and widely-used enterprise-level relational database management system suitable for handling large amounts of data with high availability and security. The NuGet Package for Oracle Database is **Oracle.EntityFrameworkCore**
- **MongoDB:** A NoSQL document-based database known for its scalability and flexibility, often used in applications requiring high performance and scalability. The NuGet Package for MongoDB Non-Relational Database is **MongoDB.EntityFrameworkCore**

For the complete list of EF Core Database Providers, please visit the following URL: <https://learn.microsoft.com/en-us/ef/core/providers/>

Why We Need to Use Entity Framework Core over EF 6.x?

Entity Framework Core offers several advantages over the older Entity Framework 6.x. They are as follows:

- **Cross-Platform Support:** EF Core is designed to be cross-platform, while EF 6.x is limited to Windows.
- **Performance Improvements:** EF Core is optimized for performance, with features like batching of SQL commands and better tracking of changes.
- **Modular Design:** EF Core is more modular, allowing developers to include only the necessary components in the applications they need.
- **Better Support for Newer .NET Features:** EF Core is designed to work with the latest .NET versions, including .NET 6, .NET 7, .NET 8, and beyond.
- **Improved Migrations:** EF Core provides a robust migrations system.
- **Continuous Development:** EF Core is actively maintained and receives regular updates, including new features and bug fixes, while EF 6.x is in maintenance mode.

For a better understanding between the EF and EF Core, please look at the following image:

Feature	EF Core	EF 6.x
Cross-Platform	Yes (Windows, macOS, Linux)	No (Windows only)
Performance	Optimized for performance	Less optimized
Modularity	Lightweight and modular	Monolithic
LINQ Support	Improved LINQ support with new features	Traditional LINQ support
Tracking Changes	Efficient change tracking	More resource-intensive tracking
Migrations	Fluent migrations with improved capabilities	Traditional migrations
Batching	Supports command batching	Does not support command batching
Eager/Explicit/Lazy Loading	Supports all loading types, with better control	Supports all, but with performance overhead
New Features	Regular updates with new features	Limited updates, maintenance mode

For a more detailed feature-by-feature comparison, please visit the URL: <https://learn.microsoft.com/en-us/ef/efcore-and-ef6/>

Prerequisites to Learn Entity Framework Core:

To take full advantage of these Entity Framework Core Tutorials, you should have basic knowledge of C# and any database such as SQL Server, Oracle, or MySQL to understand these tutorials better. Having .NET Core, Visual Studio, and SQL Server installed on your computer is good.

For whom are these Entity Framework Core (EF Core) Tutorials?

These **Entity Framework Core (EF Core) Tutorials** are designed for Students, Beginners, Intermediate, and Professional software developers who want to learn how to use Entity Framework Core (EF Core) step-by-step in detail with Real-time examples. We will provide a hands-on approach to the subject with step-by-step program examples that will assist you in learning and putting the acquired knowledge into practice. We will also discuss behind-the-scenes stories, i.e., how Entity Framework Core works behind the scenes, how internally it generates the SQL Command, how it tracks the entities, how it handles the concurrency issues, and how it implements the transactions.

Note: If we missed any Entity Framework Core (EF Core) concept, please let us know by commenting in the comment section. We promised to write an article on that topic in this Entity Framework Core tutorials series as soon as possible.

In the next article, I will discuss [How to Install the Entity Framework Core in ASP.NET Core Application](#) using Visual Studio. In this article, I briefly Introduce the Entity Framework Core. I hope you will enjoy these Entity Framework Core (EF Core) tutorials as you progress in this Entity Framework Core (EF Core) tutorial article series.